TECHNICAL BLUEPRINT · INFRASTRUCTURE DESIGN

Agent Dev Sandbox

A secure, egress-restricted development environment for AI coding agents on Debian 13: hardened rootless Podman workspaces behind an allowlist Squid gateway, an optional KVM micro-VM tier for untrusted code, and an ephemeral CI mode — all served from one reproducible image.

Debian 13 (trixie) · Podman 5.4 · QEMU/KVM
Ephemeral CI profile · Python 3.13 · Node 24
Facts verified 2026-11-05

ONE IMAGE · TWO BOUNDARIES · ONE POLICY FILE

Table of Contents

1. Executive summary 2

2. Goals, non-goals, threat model 3

3. Reference implementation survey 4

4. Architecture decisions (ADR summary) 5

5. System design 6

6. Component specifications 8

7. Representative workload fit — budget-wiser 10

8. Security analysis 10

9. Operational characteristics, logging & audit 11

10. Design questions you will ask 12

11. Runbooks 12

12. Ephemeral CI profile 13

13. Verification as specification 15

14. Roadmap 15

1. Executive summary

The environment is a two-tier "container-first, VM-when-paranoid" design, with a third consumption mode of the same image for ephemeral CI runs.

Tier 1 — daily driver. One persistent, hardened, *rootless* Podman container per project. The container runs with zero Linux capabilities, `no-new-privileges`, user-namespace mapping (`keep-id`), and cgroup caps. It sits on an *internal* Podman network that has **no default route and no external DNS**. Egress exists through exactly one path: a rootless Squid gateway enforcing a single-file domain allowlist. Everything an agent needs — Python 3.13, Node 24, Bun, pnpm, uv, git, ripgrep, SQLite, Chromium, Playwright (Node + Python), agent-browser, and the three agent CLIs — is baked into one reproducible image.

Tier 2 — hostile workloads. A KVM micro-VM per engagement, booted from an official Debian 13 cloud image, provisioned by cloud-init, with egress enforced on BOTH sides: host-side QEMU user-net restrict=on (only an ssh hostfwd and a proxy guestfwd exist) plus nftables *inside* the VM (default-deny, proxy only — no DNS rule). Inside the VM runs the **same** container image via rootless Podman — one source of truth, two boundaries.

CI mode — same image, throwaway. A third consumption mode (§12): single-shot ephemeral containers for PR checks and batch jobs, with a per-run isolation self-audit, artifact export, walltime caps, shared package caches, and a reaper for orphans. One image, three modes — interactive workspace, ephemeral CI run, container-in-VM.

Secrets never enter images: per-workspace env files (mode 0600) are injected at container start. The verification suite (`scripts/50-verify.sh`) proves both function and isolation — including *negative* tests that must fail (direct egress, external DNS) — and its output is the evidence standard for "done".

All external facts in this document were verified against live sources on **2026-11-05** (npm registry, Docker Hub, cloud.debian.org, podman documentation, the budget-wiser repository); audit remediations were applied and re-verified on **2026-11-06** (see `references/audit-remediation.md` — validation of the external audit's 22 findings and the fixes). Version pins appear as `ARG`s in `files/Containerfile`; bump them deliberately.

2. Goals, non-goals, threat model

Goals

- Develop and test codebases you would otherwise run yourself (representative workload: nordeim/budget-wiser — Next.js 16, React 19, TypeScript strict, Prisma 6 + SQLite, Vitest, Playwright E2E, Bun ≥ 1.3 / Node ≥ 24). Untrusted code belongs in Tier 2.
- Contain the blast radius of agent-executed code: no host filesystem access beyond the project dir, no capabilities, capped resources, no arbitrary network egress.
- Self-contained: no dependency on host-installed dev tooling; workspaces are disposable and reproducible.
- Agent-usable: CLIs, browsers, and shell ergonomics work out of the box; a single scripted lifecycle for everything.
- Auditable: one file is the egress policy; every guarantee has a check.

Non-goals (see `references/hardening.md` §6): multi-tenant fleets; TLS interception of proxied traffic; defending against kernel zero-days from Tier-1 workloads (that is Tier 2's job); hiding allowed-channel data flows (e.g., `git push` to an allowed host).

Threat model — two tiers

	Tier 1 (container)	Tier 2 (micro-VM)
Use when	Building/fixing known codebases (your own, curated OSS — code you would run yourself)	Auditing/running unknown repos, untrusted builds, anything that executes arbitrary scripts
Boundary	Kernel namespaces + seccomp + cap-drop (shared kernel)	Full hardware-virtualized kernel boundary
Egress	Internal network → allowlist proxy (structural: no route exists otherwise)	Host-side QEMU restrict=on (proxy-only guestfwd) → VM nftables default-deny → host allowlist proxy — enforced on BOTH sides
Residual risk	Kernel exploit escaping to host user (mitigated: rootless, non-root, no caps); workload-user escape lands on your own account (keep-id) — hence the tier discipline	VM escape (hypervisor-class, rare); supply chain of allowlisted domains
Cost	~2 GB RAM/workspace, seconds to start	~6 GB RAM/VM, one-time 15-30 min golden build, ~30 s per overlay boot

Table 1 — Threat model by tier.

3. Reference implementation survey

The producing environment for this blueprint is itself a Debian 13 sandbox running an AI coding agent (this document was generated inside it), which doubles as a working reference of a *functional* agent toolchain:

Component	Reference version	Note
OS	Debian 13.4 "trixie"	Same distro as the target host
Python	3.13.5 (`python3.13`)	trixie ships 3.13 as `python3`
`python3.13-venv`	required, separate package	**Gotcha verified here:** `python3 -m venv` fails without it
Node.js	24.22.2 (NodeSource `node_24.x`)	Same repo used by the Containerfile
npm / pnpm	11.19.1 / 12.16.1	pnpm via corepack
Bun	1.3.14	budget-wiser's primary runtime
uv	0.13.7	standalone installer, pinned in image
git	2.47.3	
Playwright (Node)	1.57.0 → image pins 1.64.1	1.64.x matches budget-wiser
Playwright browsers	chromium-1200 + headless shell + ffmpeg (613 MB in `~/.cache/ms-playwright`)	Motivates the shared `/opt` bake + per-home seeding
agent-browser	0.41.1 (npm latest 0.41.2)	**Verified:** downloads its *own* Chrome 157 to `~/.agent-browser/browsers/` — independent of Playwright's Chromium
trixie packages (verified via `apt-cache policy`)	podman 5.4.2, passt, slirp4netns 1.2.1, crun 1.21, squid 6.13, qemu 10.0.13, virtinst 5.0, cloud-image-utils 0.33, chromium 156, uidmap	All in main trixie repos
Debian cloud image	`debian-13-genericcloud-amd64.qcow2` at `cloud.debian.org/images/cloud/trixie/latest/` (+ `SHA512SUMS`)	Tier-2 base, checksum-verified by script
Agent CLIs (npm, live)	claude-code 2.1.318 · @openai/codex 0.166.0 · opencode-ai 1.19.5	All installable via `npm -g`; native installers also exist

Table 2 — Reference implementation survey (all versions verified 2026-11-05).

4. Architecture decisions (ADR summary)

D1 — Podman (rootless) over Docker CE. Rootless Podman never needs a root daemon: container *root* maps to an unprivileged subordinate uid, so a container-root escape lands with no host permissions. Be precise about the other case: with `keep-id:uid=1000` the *workload* user `dev` IS the host user — an escape *as the workload* lands on your own account. Tier 1 is therefore for code you would run yourself; untrusted code belongs in Tier 2. No setuid daemon; lifecycle via systemd user units (Quadlet). Docker's daemon requires root or rootlesskit wrappers and adds a `docker.sock` foot-gun this design refuses to have. Podman 5.4 in trixie supports everything needed (defined networks, `--internal`, `keep-id:uid=`, Quadlet, multi-`Network=`).

D2 — Egress enforcement by *network topology*, not by env-var etiquette. `podman network create --internal` (verified against `podman-network-create(1)`) gives attached containers **no default route**, disables IP forwarding on the bridge, and makes aardvark-dns answer **only container names** (everything else NXDOMAIN). Workspaces cannot bypass the proxy even if they ignore proxy env vars — there is no route to bypass with. The gateway is dual-homed: `agent-egress` (normal NAT network) + `agent-internal` (the isolated side). An env-var-only proxy would be trivially bypassable; host-firewall approaches cannot distinguish rootless container traffic from the user's own processes. Topology wins.

D3 — Squid allowlist over deny-all+mirrors. Considered Verdaccio/devpi/apt-cacher mirrors (offline-first). Rejected as default: three caches to run and keep healthy, and agents legitimately need *docs and APIs*, not just packages. The allowlist proxy gives one policy file, full logging, and a 10-second change procedure (`30-gateway.sh allow`). Trade-off stated openly: an allowlisted domain is a standing grant — treat the file as production config.

D4 — Browsers baked once, seeded per-workspace. Two browser ecosystems must coexist (§3): Playwright's Chromium (per-version revisions) and agent-browser's own Chrome. Both are baked into `/opt` (image layers, ~2 GB unpacked, shared by every workspace) and *symlinked* into each workspace's writable home by an idempotent seeder. Projects pinning other Playwright versions download their revision through the proxy into their home volume — self-healing, no copy overhead for the common case.

D5 — `--userns=keep-id:uid=1000,gid=1000`. Rootless containers otherwise remap the host user to container-root, making bind-mounted project dirs unowned by the workspace user. `keep-id:uid=1000` pins host user ↔ container `dev` (uid 1000) regardless of the host uid, so `~/agent-workspaces/<name>` is transparently shared (IDE-editable on the host) and writable in-container. This also eliminates git's "dubious ownership" class of failures.

D6 — One image, two tiers (container-in-VM). The Tier-2 VM runs rootless Podman *inside* it and loads the *same* `agent-dev-sandbox:1` image (transferred during the one-time golden build).

Toolchain drift between tiers becomes impossible; only the enforcement point differs (VM nftables instead of network topology). The CI profile extends the same principle: `files/ci-setup.sh` is bind-mounted per run, so the image itself stays byte-identical across all three consumption modes.

D7 — No sudo inside workspaces. A missing system package is an image bug: rebuild (minutes, cached). Granting sudo would re-open privilege escalation that cap-drop and no-new-privileges just closed. Language-level installs (npm/pip/uv) are unaffected.

D8 — Version-pinned ARGs, digest-pinned base. Every tool in the image is pinned to a version verified on 2026-11-05 (bun via a pinned installer tag); the base image is digest-pinned and the NodeSource repo key is fingerprint-verified in-build. Honest scope: top-level versions, the base digest, bun and the NodeSource key are pinned — the uv/bun installer scripts and transitive npm dependencies are fetched over TLS from trusted endpoints at build time and are not checksum-verified; browser revisions are resolved at build. Refresh = bump ARG + rebuild + re-verify; never `@latest` in the image.

Alternatives considered and rejected: gVisor/Kata (not in Debian repos; adds a kernel-interposition layer with its own trust base — reasonable future option, documented in hardening §4); Firecracker/Cloud Hypervisor (excellent for fleets; QEMU+KVM is packaged, scriptable and sufficient for single-user hosts — `boot_vm()` is the only function that would change); full Docker-in-Docker (defeats isolation); remote dev containers (network dependency, opposite of self-contained).

5. System design

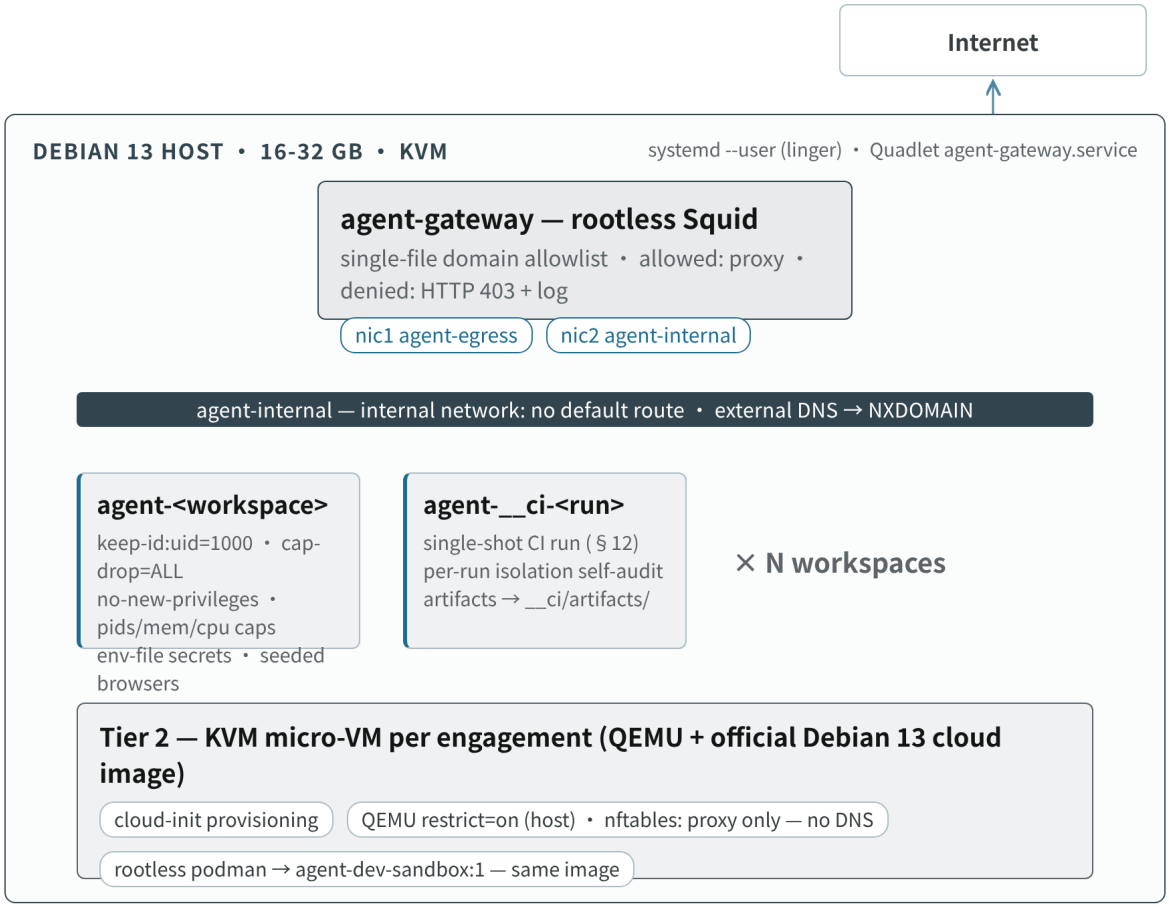


Figure 1 — System design: one host, two enforcement tiers, three consumption modes of one image.

Life of an egress request (the guarantee, step by step). A workspace process runs `bun install`. Bun reads `https_proxy` (injected as container env) and sends `CONNECT registry.npmjs.org:443` to `agent-gateway:3128` — the name resolves via aardvark-dns to the gateway's `agent-internal` interface, because that is the only resolver reachable. Squid matches the CONNECT target against the allowlist file (bind-mounted read-only from the skill bundle): allowed → Squid itself resolves DNS and proxies the TLS tunnel (payload never decrypted); denied → HTTP 403 plus one access-log line. If the process ignores proxy variables entirely and attempts a direct socket connection, the packet is dropped at the network layer: `agent-internal` has no default route and forwarding is disabled on the bridge. If it tries to smuggle a hostname past policy, there is no external DNS to resolve it with — aardvark answers container names only, everything else NXDOMAIN.

> Four independent mechanisms, one policy file: no route, NXDOMAIN, proxy 403, and full per-request logging.

Workspace model (per-project, persistent). A workspace = container + bind-mounted code dir + named home volume + env file. Stopping preserves everything; `restart` re-creates the container (re-reading the env file — key rotation without volume surgery); `snapshot` commits the container fs

(env-file keys blanked — see §6) and tars the home volume into `~/agent-workspaces/snapshots/` (0700); `destroy` removes the container, keeps code and home unless explicitly purged. Credentials for the agent CLIs live in the home volume, so logins survive container re-creation.

Tier-2 lifecycle. `base` (once): download + SHA512-verify the official cloud image → golden overlay → cloud-init (packages via the host proxy, the nftables ruleset, subuid ranges, linger, package upgrade) → transfer + `podman load` the agent image → confirmed shutdown → freeze (refusing to replace the golden while overlay VMs exist). `create <name>` (seconds): copy-on-write overlay of golden + boot + container start inside; `verify <name>` asserts the Tier-2 isolation guarantees inside the VM. Backing-file sharing makes overlays instant; the flip side — a poisoned golden poisons all overlays — is why hardening §4 recommends rebuilding golden per trust boundary.

6. Component specifications

Image contents (`files/Containerfile`, all pins verified 2026-11-05): Debian trixie-slim base (digest-pinned); Python 3.13 + venv + dev headers; Node 24 (NodeSource, key fingerprint verified); Bun 1.3.14 (pinned installer tag); pnpm 12; uv 0.13.7; git/git-lfs; ripgrep, fd, jq, tmux, socat, openssh-client; build-essential + libssl/zlib/sqlite headers; SQLite; system Chromium 156 + fonts; Playwright 1.64.1 (Node *and* Python, shared browser revisions) baked to `/opt/ms-playwright`; agent-browser 0.41.2 with its Chrome baked to `/opt/agent-browser-data`; claude-code 2.1.318, codex 0.166.0, opencode 1.19.5; `NODE_USE_ENV_PROXY=1`; non-root `dev` (uid 1000); `sleep infinity` as PID 1. Result: ~3.5-4 GB image (~2 GB of it the unpacked `/opt` browser bake; the reference environment's Playwright-only cache was 613 MB), ~2 GB RAM per active workspace at idle-to-moderate load.

Layer	Pinned version	Why it is there
python3.13 + venv + dev	trixie 3.13.5	system Python for the distro; venv package is the documented trixie gotcha
nodejs	24.x (NodeSource `node_24.x`)	budget-wiser floor is Node ≥ 24; matches reference environment
bun	1.3.14 (`BUN_VERSION` ARG, pinned installer tag)	budget-wiser's primary runtime (≥ 1.3)
pnpm	12.16.1	package manager of choice for agent workflows
uv	0.13.7	fast per-project Python envs; standalone binary
playwright (node+py)	1.64.1	matches budget-wiser's pin → shared browser revision, no re-download
agent-browser	0.41.2	accessibility-snapshot browser driving for agents (own Chrome 157)
claude-code / codex / opencode	2.1.318 / 0.166.0 / 1.19.5	the three agent CLIs, all `npm -g` installable, no Node runtime needed at their own runtime
chromium	trixie 156	system browser + shared library base for both browser ecosystems
ripgrep/fd/jq/socat/tmux	trixie packaged	agent ergonomics (search, JSON, port/probe, sessions)

Table 3 — Image layers and their justification.

Egress policy (`files/squid/allowlist.txt`): Debian mirrors; runtime installers (NodeSource, nodejs.org, astral.sh, bun.sh); registries (npm, yarn, PyPI); git hosts (GitHub, GitLab — leading-dot entries cover subdomain CDNs); Prisma engine binaries (`binaries.prisma.sh`, needed by the representative workload); Playwright CDN; the AI provider APIs; two verification domains. Squid `dstdomain` semantics: a leading dot matches subdomains, a bare domain is an EXACT match, `*.` is not a wildcard and is rejected by `allow`. Squid resolves DNS (workspaces never need it). Adding a domain: one command, both tiers, restart included — and every change is `squid -k parse`-validated before the gateway restarts.

Ephemeral CI profile (`scripts/45-ci-run.sh` + `files/ci-setup.sh`): single-shot containers from the same image — CI env, shared package caches, per-run isolation self-audit, artifact export, walltime timeout, orphan reaper. Full design in §12.

Resource budget (16-32 GB host): gateway ~50 MB; each Tier-1 workspace default 6 GB/4 CPU cap (per-container) inside a user-slice budget; one Tier-2 VM 6 GB/4 vCPU by default. Comfortable: 2-4 active workspaces + 1 VM on 32 GB; 1-2 workspaces on 16 GB.

Secrets plumbing. Host file `~/.config/agent-sandbox/workspaces/<n>.env` (0600, never committed, never in an image) → `--env-file` at `podman run`. Agent CLI auth tokens live inside the home volume only. Snapshot caveat (verified against podman 5.4 source): `podman commit` copies the

container's environment into the image config — `40-workspace.sh snapshot` therefore blanks every env-file key from the committed image; rotate keys after snapshotting a workspace that had live secrets, and never distribute `ws-snap-*` images. Tier-2 env files live inside the VM (`~/.config/agent-sandbox/`), so host compromise of the env dir does not automatically include VM workspaces' keys.

7. Representative workload fit — budget-wiser

The chosen reference repo stresses every axis of the design: Next.js 16 + React 19 need Node ≥ 24 (or Bun ≥ 1.3) — both provided; Prisma 6 + SQLite needs libssl/sqlite — provided; `bun install` needs registry.npmjs.org — allowlisted — and, because budget-wiser enables Prisma's install scripts, it also fetches engine binaries from `binaries.prisma.sh` (allowlisted for exactly that reason; run the acceptance path to confirm the full domain set); `db:push/db:seed` are local file operations once the engines are present; Vitest runs offline; Playwright E2E launches Chromium against `localhost` — covered by `no_proxy` and the baked browser; `agent-browser` verifies UI state via accessibility snapshots. Onboarding inside a workspace:

```
scripts/40-workspace.sh create wiser --repo https://github.com/nordeim/budget-wiser.git
scripts/40-workspace.sh shell wiser
# in-workspace:
bun install && bun run db:push && bun run db:seed && bun run gate   # lint+type+unit+build+smoke+e2e
bun run dev &       # localhost:3000
agent-browser open http://localhost:3000 && agent-browser snapshot -i
bun run test:e2e    # Playwright, local server — no external egress needed
```

`50-verify.sh` performs the first step (clone via proxy) as a standing check; the rest is the documented user acceptance path.

8. Security analysis

Guarantees enforced by construction: workspace processes run as non-root with zero capabilities, no privilege escalation, and cgroup caps; the only route off the host user's network namespace is Squid with a domain allowlist (403 + logged for everything else); external DNS is unavailable; no host path is writable inside a workspace except the project dir. Be honest about what that means: the bind mount shares YOUR uid, so a workload-user escape reaches your own files everywhere — not just the project dir. That is the same trust level as running the code by hand, which is precisely the Tier-1 contract; code you do not trust that far belongs in Tier 2. Host-executed persistence is also in scope: payloads planted in `.git/hooks` or package scripts run the next time YOU run host tooling against the mount — treat the mount as workload-owned. Tier-2 adds a full VM boundary and keeps all host paths out entirely.

Verified, with caveats stated: rootless defined-network NAT egress is a standard podman 5 capability and the `--internal` semantics are documented (netavark backend — preflight --post checks it); if rootless NAT misbehaves, debug netavark/pasta — there is deliberately NO rootful-gateway fallback, because workspaces on an `--internal` network cannot reach host loopback by design, and moving them to a routed network would break invariant 3. Chromium-family browsers do not reliably honor proxy env vars in every launch mode — external-page browsing via agent-browser is therefore a SOFT check with the explicit per-project Playwright proxy option documented; local dev-server testing (the dominant agent flow) is unaffected and hard-verified.

Known residual risks (by tier, see §2 table and hardening §6): shared kernel for Tier 1; workload-user escape = host user (keep-id; the tier discipline is the answer); host-side persistence vectors via the shared mount (`.git/hooks` etc.); supply chain of allowlisted domains AND of build-time endpoints (installers, npm transitives — see D8); allowed-channel exfiltration; side channels on shared hardware. The design's answer to the first two is the tier switch, to the third is allowlist governance + your normal package review, to the rest is stated scope.

9. Operational characteristics, logging & audit

Timing profile (mid-range host): image build 5-15 min (first) / seconds (cached); workspace create ~3 s + first `seed-playwright-browsers` (< 1 s); workspace `restart` ~2 s; gateway start ~1 s; Tier-2 golden build 15-30 min (one-time); Tier-2 overlay create ~40 s to SSH. A CI run adds ~5-8 s of overhead around the command itself (container create + browser seeding + isolation self-audit); artifact copying is size-dependent.

What happens on reboot: linger + the gateway Quadlet bring the proxy back automatically (the generator applies `[Install]` at daemon-reload — `systemctl --user enable` is never used on generated units). Workspaces have NO restart policy: after a host reboot every workspace is stopped, and `40-workspace.sh start <name>` resumes it — a deliberate choice: agents should not auto-resume unfinished work unattended. VMs do not auto-start (their qemu processes are user-managed, not systemd units — keep it that way unless you add units yourself).

Audit trail: Squid access logs are the egress audit record — one line per proxied request and per denial. Container path: `journalctl --user -u agent-gateway.service` (or `podman logs agent-gateway`); Tier-2 host path: `/var/log/squid/access.log`. Answering "did workspace X contact domain Y?" is a one-line grep. Retention is your host's journald/logrotate policy — size the logs to your compliance needs.

Host maintenance: Debian security updates apply normally (host packages are outside all sandboxes). After a host kernel update + reboot, re-run `50-verify.sh` before resuming untrusted work — the suite is cheap and the kernel is the Tier-1 boundary.

10. Design questions you will ask

Why not Claude Code's own sandbox / Codex's built-in sandboxing? Both CLIs ship process-level sandboxes (seccomp/Landlock/namespace wrappers). They harden *the CLI's own tool calls* — but the user may run other agents, ad-hoc commands, or a different CLI version; and process sandboxes do not provide resource caps, egress allowlists, or a reproducible environment. Caveat: those sandbox modes generally need privileges this container deliberately drops, so expect them to run in a degraded or disabled mode INSIDE a workspace — the workspace boundary is the one that counts, and agent-internal sandboxing remains an optional extra layer, never the boundary.

Why is the gateway a container instead of host squid for Tier 1? Rootless containers on an internal network cannot reach host loopback; the dual-homed gateway container keeps the entire Tier-1 plane inside the unprivileged user realm (no root-owned moving parts) and makes the whole thing portable via Quadlet. Tier 2 does use host squid — VMs reach it via QEMU's 10.0.2.2 mapping, which lands on loopback.

Why allow example.com but not a general docs allowlist? Every line is a standing grant to untrusted code. Two inert verification domains are shipped; anything else is a deliberate, reviewable policy decision via `30-gateway.sh allow`. Agents that need docs access get it the same way — explicitly, observably, revocably.

Can a workspace use Docker/Compose inside? No — deliberately. Nested container runtimes need privileges this design refuses to grant. Projects needing multi-service topologies should use podman-in-VM (Tier 2, where rootless podman IS available inside the VM) or the host-side scripts.

11. Runbooks

Day-0 (install, ~25-40 min including image build): `00-preflight.sh` → `10-host-bootstrap.sh [--tier2]` → re-login if kvm group added → `00-preflight.sh --post` → `20-build-image.sh` → `30-gateway.sh up` → `50-verify.sh` → (optional) `60-tier2-microvm.sh base`.

Day-1 (new project): `40-workspace.sh create <name> [--repo URL]` → edit env file → `restart <name>` → `shell <name>` → run the agent CLI.

Day-2 (ongoing): allowlist changes (`30-gateway.sh allow`); tool refresh (bump ARG → rebuild → re-verify → `restart` workspaces); snapshots before risky agent sessions; `podman system df` housekeeping; allowlist review; rebuild Tier-2 golden between trust boundaries.

12. Ephemeral CI profile

The roadmap's "ephemeral CI profiles" item, delivered. The insight that keeps it cheap: CI here is not a new environment but a new *consumption mode* of the one that already exists.

Mode	Tool	Lifecycle	Home	Code
Interactive workspace	`40-workspace.sh`	persistent (stop/start/restart)	named volume `agent-home-<n>`	host bind `~/agent-workspaces/<n>`
Ephemeral CI run	`45-ci-run.sh`	one command, then destroyed	container layer (or run-local bind with `--read-only-rootfs`)	run dir, or `--src` checkout
Container-in-VM	`60-tier2-microvm.sh`	per engagement	inside the VM	VM disk

Table 4 — One image, three consumption modes.

Runner. `45-ci-run.sh run [--repo URL | --src DIR] [--timeout S] -- <command...>` creates a single-shot container that is hardened *exactly* like a workspace — keep-id, cap-drop=ALL, no-new-privileges, `agent-internal` only, env-file secrets, pids/memory/cpu caps — plus a walltime `timeout` (default 3600 s). The command streams live *and* is captured to `artifacts/console.log` (repository URLs are redacted before they reach any log or summary). Teardown always follows the run (`--keep-on-failure` opts out for debugging); the post-run artifact copier is symlink-contained (`cp -rP` + realpath checks) because the run directory is workload-controlled; `reap` removes orphans and stale run dirs past a TTL; `list` prints past runs from their summaries; `cache` wipes the shared package caches. `jq` on the host is required (bootstrap installs it; the runner checks upfront).

In-container preamble. `files/ci-setup.sh` is bind-mounted in (image unchanged — D6 holds): it sets `CI=true` (pnpm then defaults to a frozen lockfile in CI), points npm/pip/uv/bun/pnpm caches at the shared tree, seeds browsers, optionally shallow-clones `CI_REPO_URL`, then — before any workload code runs — **audits isolation**: uid 1000, zero caps, NoNewPrivs, direct egress must fail, external DNS must fail, the proxy must 403 a known-denied domain and 200 an allowlisted one. Results land in `artifacts/isolation-audit.json`; any failure exits **66** without running the command. Every CI run therefore carries its own evidence that the sandbox guarantees held *for that run*.

Artifacts. `~/agent-workspaces/__ci/artifacts/<run-id>/` keeps `console.log`, `isolation-audit.json`, a schema'd `run-summary.json` (exit code, duration, audit status, cache mode, copied files), and the report dirs the copier knows by default (`playwright-report`, `test-results`, `coverage`, `dist`, `build`, `junit.xml`; extend with `--artifact-path`, globs allowed).

Caches and trust. Shared caches are a host bind mount (`~/agent-workspaces/__ci/cache` → `/cache`) — deliberately *not* a named volume, because same-uid bind mounts are this design's proven primitive (D5); no orphan volumes can ever exist. The tree is shared writable state across runs: pnpm's own documentation makes exactly this point about its store, and the same reasoning applies to npm/pip/uv/bun caches. Same host, same user, same trust domain — fine for your own projects; `--no-cache-vol` gives a run cold, isolated caches when the source is less trusted, and hostile PR builds belong in Tier 2 (the same script runs inside the VM). CI mode changes consumption, never the trust tier.

Naming. Containers are `agent-__ci-<run-id>`, state lives under the `__ci/` subtree of the workspaces root. `40-workspace.sh`'s `validate_name` reserves the `__` prefix, so CI containers cannot collide with interactive workspaces and `reap` cannot touch them — the same structural reservation the verify canary (`__verify`) uses.

Representative run (budget-wiser gate):

```
scripts/45-ci-run.sh run --repo https://github.com/nordeim/budget-wiser.git \
    -- bash -lc 'bun install && bun run db:push && bun run db:seed && bun run gate'
scripts/45-ci-run.sh list
```

Exit codes are a contract:

Exit	Meaning
0	ok — command succeeded, self-audit passed
66	isolation audit failed — guarantees did not hold; command never ran
124	walltime timeout reached
125 / 126 / 127	podman-level error (image missing, name clash, runtime failure)
any other	the command's own exit code, propagated

Table 5 — CI run exit-code contract.

See `references/troubleshooting.md` §9 for the failure playbook (including why `--skip-audit` is never a fix).

13. Verification as specification

`50-verify.sh` maps each guarantee to a check, and — crucially — encodes the *negative* tests as first-class requirements: direct egress must fail, external DNS must fail, non-allowlisted domains must 403. A green run is a reproducible evidence ledger (per-check PASS/FAIL with captured output), designed so an AI operator cannot honestly report "done" without it — and the escalation rule forbids weakening checks to force green. The CI profile extends the same idea to every single run: its self-audit is 50-verify's isolation section, in miniature, attached to each job's artifacts.

14. Roadmap

Registry mode (local `podman push` target for team-shared images); ephemeral CI profiles — **delivered** (§12); gVisor/Kata runtime option for Tier 1.5; DNS-level allowlisting in Tier 2 (dnsmasq allow-resolver); automated monthly golden rebuilds with drift reports. Each extension is a delta on this blueprint, not a redesign.